SYSTEM SPECIFICATION // EPISODE 14

I Gave JARVIS Real-Life Eyes to See My Screen! (EP 14) 🤖⚡

BY **PULSE OF AI** | Chief Architect: **Swayam Dhawale**

> ⚠ **CRITICAL PREREQUISITE & RUNTIME ADVISORY**
>
> JARVIS is a modular neural ecosystem. **You MUST watch all preceding episodes (Parts 1–13) and extract the required source scripts, environment configurations, and dependency libraries directly from their pinned comments**. Attempting to run Episode 14 independently without prior audio loops, hardware telemetry bridges, and Gemini Live websocket pipelines will cause fatal runtime errors.

// COMPLETE SYSTEM ROADMAP (PARTS 1-13 ARCHIVE)

Part 1: Core Setup & Environment Pipeline	Part 8: Hardware Telemetry & Silicon Sensors
Part 2: Bilingual Real-Time Voice Engine	Part 9: Deep Task Optimizer & Taskkill Fallback
Part 3: Ultra-Low Latency Websocket Loop	Part 10: Dual Streaming Full-Duplex Engine
Part 4: Neural Speech Stream & Audio Buffer	Part 11: Dynamic Audio Ducking & Volume Attenuation
Part 5: 70+ Global Languages Layer	Part 12: GSMTC Active Media & Playback Inspector
Part 6: Autonomous Browser & DOM Hub	Part 13: Zero-Touch YouTube Timeline & Shorts Control
Part 7: Zero-Click App Controller & UWP Hooks	Official Hub: Pulse of AI Official Channel

📁 MODULE 1: software_control.py (Universal Process, Media & GSMTC Controller)

```
"""
software_control.py
Universal Application, Deep Task Purge & Full Media/Browser Navigator for JARVIS (Episode 14)
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import os
import sys
import ctypes
import subprocess
import webbrowser
import asyncio
import psutil
import pyautogui

# Win32 GUI & Window Messaging
try:
    import win32gui
    import win32con
    import win32process
except ImportError:
    win32gui = None

# WinRT / Windows GSMTC Imports
try:
    from winsdk.windows.media.control import (
        GlobalSystemMediaTransportControlsSessionManager as MediaManager
    )
except ImportError:
    MediaManager = None

# Key Codes
VK_MEDIA_NEXT_TRACK = 0xB0
VK_MEDIA_PREV_TRACK = 0xB1
VK_MEDIA_STOP       = 0xB2
VK_MEDIA_PLAY_PAUSE = 0xB3
VK_VOLUME_MUTE      = 0xAD
VK_K                = 0x4B  # YouTube play/pause
VK_DOWN             = 0x28  # Scroll down / Next Short
VK_UP               = 0x26  # Scroll up / Prev Short
VK_LEFT             = 0x25  # Rewind 5s
VK_RIGHT            = 0x27  # Forward 5s
VK_J                = 0x4A  # Rewind 10s
VK_L                = 0x4C  # Fast forward 10s
VK_F                = 0x46  # Fullscreen
VK_T                = 0x54  # Theater mode
KEYEVENTF_EXTENDEDKEY = 0x0001
KEYEVENTF_KEYUP       = 0x0002

# Common process alias map for accurate Windows termination
```

```
APP_PROCESS_MAP = {
    "vs code": "code",
    "vscode": "code",
    "visual studio code": "code",
    "chrome": "chrome",
    "google chrome": "chrome",
    "edge": "msedge",
    "microsoft edge": "msedge",
    "firefox": "firefox",
    "brave": "brave",
    "spotify": "spotify",
    "discord": "discord",
    "notepad": "notepad",
    "calculator": "calculatorapp",
    "calc": "calculatorapp",
    "terminal": "windowsterminal",
    "task manager": "taskmgr",
    "word": "winword",
    "excel": "excel",
}

def _send_hardware_media_key(key_code: int):
    """Sends native hardware media key press."""
    ctypes.windll.user32.keybd_event(key_code, 0, KEYEVENTF_EXTENDEDKEY, 0)
    ctypes.windll.user32.keybd_event(key_code, 0, KEYEVENTF_EXTENDEDKEY | KEYEVENTF_KEYUP, 0)

def _find_browser_windows():
    """Finds all open browser window handles and titles."""
    browser_windows = []
    target_exes = ["chrome.exe", "msedge.exe", "brave.exe", "firefox.exe"]

    if win32gui is None:
        return browser_windows

    def enum_windows_callback(hwnd, extra):
        if win32gui.IsWindowVisible(hwnd):
            title = win32gui.GetWindowText(hwnd).strip()
            if title:
                try:
                    _, pid = win32process.GetWindowThreadProcessId(hwnd)
                    p = psutil.Process(pid)
                    if p.name().lower() in target_exes:
                        browser_windows.append((hwnd, p.name().lower(), title))
                except Exception:
                    pass
        return True

    win32gui.EnumWindows(enum_windows_callback, None)
    return browser_windows

def _send_key_to_browser(key_code: int) -> bool:
    """Sends a keystroke directly to background browser window without stealing focus."""
    windows = _find_browser_windows()
    if not windows or win32gui is None:
        return False

    target_hwnd = None
    for hwnd, name, title in windows:
        if "youtube" in title.lower() or "shorts" in title.lower():
            target_hwnd = hwnd
            break

    if not target_hwnd and windows:
        target_hwnd = windows[0][0]

    if target_hwnd:
        win32gui.PostMessage(target_hwnd, win32con.WM_KEYDOWN, key_code, 0)
        win32gui.PostMessage(target_hwnd, win32con.WM_KEYUP, key_code, 0)
        return True
    return False

def _send_browser_play_pause_background() -> bool:
    """Sends YouTube's native 'k' toggle key directly to browser window without stealing focus."""
    windows = _find_browser_windows()
    if not windows or win32gui is None:
        return False

    target_hwnd = None
    for hwnd, name, title in windows:
        if "youtube" in title.lower() or "shorts" in title.lower():
            target_hwnd = hwnd
            break

    if not target_hwnd and windows:
        target_hwnd = windows[0][0]

    if target_hwnd:
        win32gui.PostMessage(target_hwnd, win32con.WM_KEYDOWN, VK_K, 0)
        win32gui.PostMessage(target_hwnd, win32con.WM_KEYUP, VK_K, 0)
        win32gui.PostMessage(target_hwnd, win32con.WM_APPCOMMAND, 0, 0xE0000)
        return True
```

```
    return False

# --- Real-Time Active Playback State Inspector ---
async def _check_media_playing_async() -> bool:
    """Returns True ONLY if audio/video is actively in a PLAYING state via GSMTC."""
    if MediaManager is None:
        return False
    try:
        manager = await MediaManager.request_async()
        session = manager.get_current_session()
        if not session:
            sessions = manager.get_sessions()
            if sessions and len(sessions) > 0:
                session = sessions[0]

        if session:
            info = session.get_playback_info()
            if info:
                # 4 corresponds to GlobalSystemMediaTransportControlsSessionPlaybackStatus.PLAYING
                status = int(info.playback_status)
                return status == 4
    except Exception:
        pass
    return False

def is_media_actively_playing() -> bool:
    """Synchronous check to detect if background media is currently playing."""
    try:
        loop = asyncio.new_event_loop()
        asyncio.set_event_loop(loop)
        result = loop.run_until_complete(_check_media_playing_async())
        loop.close()
        return bool(result)
    except Exception:
        return False

# --- Episode 13: YouTube Navigation Tools ---
def navigate_shorts(direction: str = "next") -> str:
    """Scrolls YouTube Shorts or web feed hands-free."""
    cmd = direction.lower().strip()
    if cmd in ["next", "down", "forward"]:
        if _send_key_to_browser(VK_DOWN):
            return "Skipped to next Short, sir."
        pyautogui.press("down")
        return "Scrolled to next Short, sir."
    elif cmd in ["previous", "prev", "up", "back"]:
        if _send_key_to_browser(VK_UP):
            return "Returned to previous Short, sir."
        pyautogui.press("up")
        return "Scrolled to previous Short, sir."
    return "Direction must be 'next' or 'previous', sir."

def skip_video(direction: str = "next") -> str:
    """Skips to next or previous video in a YouTube playlist/queue."""
    cmd = direction.lower().strip()
    if cmd in ["next", "forward", "skip"]:
        pyautogui.hotkey("shift", "n")
        return "Playing next video, sir."
    elif cmd in ["previous", "prev", "back"]:
        pyautogui.hotkey("shift", "p")
        return "Playing previous video, sir."
    return "Invalid direction, sir."

def seek_video(action: str = "forward") -> str:
    """Seeks or fast-forwards/rewinds YouTube video by 10 seconds."""
    cmd = action.lower().strip()
    if cmd in ["forward", "fast_forward", "ahead"]:
        if _send_key_to_browser(VK_L):
            return "Advanced 10 seconds, sir."
        pyautogui.press("l")
        return "Skipped 10 seconds forward, sir."
    elif cmd in ["rewind", "back", "backward"]:
        if _send_key_to_browser(VK_J):
            return "Rewound 10 seconds, sir."
        pyautogui.press("j")
        return "Rewound 10 seconds, sir."
    return "Action must be 'forward' or 'rewind', sir."

def toggle_video_view(mode: str = "fullscreen") -> str:
    """Toggles Fullscreen or Theater mode in YouTube."""
    cmd = mode.lower().strip()
    if cmd in ["fullscreen", "full_screen", "full"]:
        if _send_key_to_browser(VK_F):
            return "Fullscreen toggled, sir."
        pyautogui.press("f")
        return "Fullscreen mode toggled, sir."
    elif cmd in ["theater", "theatre", "cinema"]:
        if _send_key_to_browser(VK_T):
            return "Theater mode toggled, sir."
        pyautogui.press("t")
        return "Theater mode toggled, sir."
```

```
    return "Mode must be 'fullscreen' or 'theater', sir."

# --- Native Async Media Controller ---
async def control_media(action: str) -> str:
    """Controls background system media playback (Spotify, YouTube, VLC)."""
    cmd = action.lower().strip().replace(" ", "_")

    session = None
    if MediaManager is not None:
        try:
            manager = await MediaManager.request_async()
            session = manager.get_current_session()
            if not session:
                sessions = manager.get_sessions()
                if sessions and len(sessions) > 0:
                    session = sessions[0]
        except Exception:
            session = None

    if session:
        try:
            if cmd in ["play", "pause", "play_pause", "resume", "toggle"]:
                await session.try_toggle_play_pause_async()
                return "Playback toggled, sir."
            elif cmd in ["next", "skip", "next_track"]:
                await session.try_skip_next_async()
                return "Skipped to next track, sir."
            elif cmd in ["previous", "prev", "previous_track", "back"]:
                await session.try_skip_previous_async()
                return "Playing previous track, sir."
            elif cmd in ["stop"]:
                await session.try_stop_async()
                return "Playback stopped, sir."
        except Exception:
            pass

    if cmd in ["play", "pause", "play_pause", "resume", "toggle"]:
        if _send_browser_play_pause_background():
            return "Toggled browser video playback, sir."
        _send_hardware_media_key(VK_MEDIA_PLAY_PAUSE)
        return "Playback toggled via hardware key, sir."
    elif cmd in ["next", "skip", "next_track"]:
        _send_hardware_media_key(VK_MEDIA_NEXT_TRACK)
        return "Skipped track, sir."
    elif cmd in ["previous", "prev", "previous_track", "back"]:
        _send_hardware_media_key(VK_MEDIA_PREV_TRACK)
        return "Previous track, sir."
    elif cmd in ["mute", "unmute", "toggle_mute"]:
        _send_hardware_media_key(VK_VOLUME_MUTE)
        return "Audio mute toggled, sir."

    return "No active media stream found to control, sir."

async def get_current_media_info() -> str:
    """Inspects active background media (Spotify, YouTube in Chrome/Edge, VLC) and returns track title."""
    if MediaManager is not None:
        try:
            manager = await MediaManager.request_async()
            session = manager.get_current_session()
            if not session:
                sessions = manager.get_sessions()
                if sessions and len(sessions) > 0:
                    session = sessions[0]

            if session:
                props = await session.try_get_media_properties_async()
                if props and props.title and len(props.title.strip()) > 0:
                    title = props.title.strip()
                    artist = props.artist.strip() if props.artist else ""
                    if artist:
                        return f"Currently playing '{title}' by {artist}, sir."
                    return f"Currently playing '{title}', sir."
        except Exception:
            pass

    windows = _find_browser_windows()
    for _, _, raw_title in windows:
        clean = raw_title.lower()
        if "youtube" in clean or "shorts" in clean:
            clean_title = raw_title
            for suffix in ["- Google Chrome", "- Microsoft Edge", "- Brave", "- YouTube"]:
                clean_title = clean_title.replace(suffix, "").strip()

            if clean_title.startswith("(") and ")" in clean_title:
                clean_title = clean_title.split(")", 1)[1].strip()

            if clean_title and clean_title.lower() != "youtube":
                return f"Currently playing YouTube video: '{clean_title}', sir."

    for proc in psutil.process_iter(['name']):
        try:
```

```
            if proc.name().lower() == "spotify.exe":
                for _, name, title in windows:
                    if name == "spotify.exe" and title and title.lower() != "spotify":
                        return f"Currently playing on Spotify: '{title}', sir."
        except Exception:
            pass

    return "No active media stream or video detected, sir."

# --- App Launch & Process Management Tools ---
def open_website(url: str) -> str:
    clean_url = url.strip()
    if not clean_url.startswith(("http://", "https://")):
        clean_url = "https://" + clean_url
    webbrowser.open(clean_url)
    return f"Opened {clean_url}"

def open_application(app_name: str) -> str:
    query = app_name.lower().strip()
    uwp_protocols = {
        "camera": "microsoft.windows.camera:", "webcam": "microsoft.windows.camera:",
        "calculator": "calculator:", "calc": "calculator:", "photos": "ms-photos:",
        "settings": "ms-settings:", "store": "ms-windows-store:", "spotify": "spotify:",
        "paint": "mspaint", "notepad": "notepad", "cmd": "cmd", "terminal": "wt",
        "explorer": "explorer", "task manager": "taskmgr", "taskmgr": "taskmgr",
    }
    if query in uwp_protocols:
        try:
            target = uwp_protocols[query]
            if target.endswith(":"):
                os.system(f"start {target}")
            else:
                subprocess.Popen(target, shell=True)
            return f"Launched {app_name}"
        except Exception:
            pass

    if sys.platform == "win32":
        search_dirs = [
            r"C:\ProgramData\Microsoft\Windows\Start Menu\Programs",
            os.path.expandvars(r"%APPDATA%\Microsoft\Windows\Start Menu\Programs"),
            os.path.expandvars(r"%LOCALAPPDATA%\Programs"),
        ]
        matches = []
        for directory in search_dirs:
            if not os.path.exists(directory):
                continue
            for root, _, files in os.walk(directory):
                for file in files:
                    if file.lower().endswith((".lnk", ".exe", ".url")):
                        file_name = file.lower().rsplit(".", 1)[0]
                        if query == file_name:
                            os.startfile(os.path.join(root, file))
                            return f"Launched {app_name}"
                        elif query in file_name:
                            matches.append(os.path.join(root, file))
        if matches:
            os.startfile(matches[0])
            return f"Launched {app_name}"

        try:
            ps_cmd = (
                f"$app = Get-StartApps | Where-Object {{ $_.Name -like '*{query}*' }} | Select-Object -First 1; "
                f"if ($app) {{ Start-Process explorer.exe -ArgumentList ('shell:AppsFolder\\' + $app.AppID) }}"
            )
            subprocess.Popen(["powershell", "-NoProfile", "-WindowStyle", "Hidden", "-Command", ps_cmd])
            return f"Launched {app_name}"
        except Exception as e:
            return f"Failed to launch {app_name}: {str(e)}"
    else:
        try:
            subprocess.Popen([query])
            return f"Launched {app_name}"
        except Exception as e:
            return f"Failed to launch {app_name}: {str(e)}"

def close_application(app_name: str) -> str:
    """Closes an active Windows application cleanly and forcefully."""
    if not app_name:
        return "No application name specified to close, sir."

    raw_query = app_name.lower().strip().replace(".exe", "")
    query = APP_PROCESS_MAP.get(raw_query, raw_query)
    protected = {"explorer.exe", "svchost.exe", "system", "idle", "csrss.exe", "python.exe", "cmd.exe"}
    current_pid = os.getpid()
    killed = 0

    # Pass 1: psutil direct process tree termination
    for proc in psutil.process_iter(['pid', 'name']):
        try:
            pid = proc.info['pid']
```

```
            pname = (proc.info['name'] or "").lower()
            if pid == current_pid or pname in protected:
                continue
            if query == pname.replace(".exe", "") or query in pname:
                try:
                    for child in proc.children(recursive=True):
                        child.kill()
                except Exception:
                    pass
                proc.kill()
                killed += 1
        except Exception:
            continue

    if killed > 0:
        return f"Closed {app_name}, sir."

    # Pass 2: Windows native taskkill fallback
    target_exe = query if query.endswith(".exe") else f"{query}.exe"
    try:
        output = subprocess.run(
            f'taskkill /F /T /IM "{target_exe}"',
            shell=True,
            capture_output=True,
            text=True
        )
        if output.returncode == 0:
            return f"Closed {app_name}, sir."
    except Exception:
        pass

    # Pass 3: Window title fallback for UWP and renamed processes
    try:
        ps_cmd = f'Get-Process | Where-Object {{ $_.MainWindowTitle -like "*{raw_query}*" }} | Stop-Process -Force'
        ps_run = subprocess.run(["powershell", "-Command", ps_cmd], capture_output=True, text=True)
        if ps_run.returncode == 0 and not ps_run.stderr:
            return f"Closed {app_name}, sir."
    except Exception:
        pass

    return f"Could not find any active process for {app_name}, sir."

def purge_background_tasks() -> str:
    target_signatures = [
        "crashpad_handler", "googleupdate", "microsoftedgeupdate", "edgeupdate",
        "epicwebhelper", "steamwebhelper", "discordhelper", "discordcanary",
        "cortana", "gamebarpresencewriter", "nvcontainer", "adobearm"
    ]
    protected = {"explorer.exe", "svchost.exe", "system", "python.exe"}
    initial_ram = psutil.virtual_memory().used
    killed_count = 0
    killed_names = set()

    for proc in psutil.process_iter(['pid', 'name']):
        try:
            pid = proc.info['pid']
            raw_name = proc.info['name'] or ""
            lower_name = raw_name.lower()
            if pid == os.getpid() or lower_name in protected:
                continue
            if any(sig in lower_name for sig in target_signatures):
                proc.kill()
                killed_count += 1
                killed_names.add(raw_name.replace(".exe", "").title())
        except Exception:
            continue

    freed_mb = max(0, int((initial_ram - psutil.virtual_memory().used) / (1024 * 1024)))
    if killed_count > 0:
        names_str = ", ".join(list(killed_names)[:3])
        return f"Terminated {killed_count} tasks including {names_str}, freeing {freed_mb} MB RAM, sir."
    return "Background tasks are already optimized, sir."
```

📁 MODULE 2: agent.py (Gemini Live Engine with Vision & Adaptive Audio Loop)

```
"""
agent.py
Real-Time Gemini Live Engine with Hands-Free Media Navigation & Local Vision (Episode 14)
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import asyncio
import datetime
import os
import sys
import math
import struct
import pyaudio
import winsound
```

```
import keyboard
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console

# Hardware & Telemetry
from hardware_control import (
    get_brightness,
    get_volume,
    set_brightness,
    set_volume,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    restore_master_volume,
)

# Software, Media Telemetry & Navigation Tools
from software_control import (
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    is_media_actively_playing,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
)

# Local Zero-Latency Vision Tools (Powered by Qwen2.5-VL / Ollama)
from screen_vision import (
    analyze_screen,
    find_ui_element,
)

load_dotenv()

console = Console()
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()

model_is_speaking = False
media_is_playing_state = False

def calculate_rms(data: bytes) -> float:
    """Calculates RMS audio energy."""
    count = len(data) // 2
    if count == 0:
        return 0.0
    shorts = struct.unpack(f"{count}h", data)
    sum_squares = sum(s * s for s in shorts)
    return math.sqrt(sum_squares / count)

def play_sfx_chime():
    try:
        winsound.Beep(1200, 60)
        winsound.Beep(1800, 80)
    except Exception:
        pass

def play_startup_chime():
    try:
        winsound.Beep(600, 70)
        winsound.Beep(900, 70)
        winsound.Beep(1400, 120)
    except Exception:
        pass

# Master Tools Manifest (Preserves Episode 13 + Vision Additions)
tools = [
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    navigate_shorts,
    skip_video,
    seek_video,
```

```
    toggle_video_view,
    set_volume,
    get_volume,
    set_brightness,
    get_brightness,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    analyze_screen,
    find_ui_element,
]

def get_time_greeting() -> str:
    current_hour = datetime.datetime.now().hour
    if 5 <= current_hour < 12:
        return "Good morning, sir."
    elif 12 <= current_hour < 17:
        return "Good afternoon, sir."
    else:
        return "Good evening, sir."

# Background monitor: updates media state every second
async def monitor_playback_state():
    global media_is_playing_state
    last_state = None
    while True:
        try:
            current_state = await asyncio.to_thread(is_media_actively_playing)
            if current_state != last_state:
                media_is_playing_state = current_state
                last_state = current_state
                if current_state:
                    console.print("\n[bold yellow]🛡️ [MEDIA PLAYING DETECTED] Push-to-Talk Armed: Hold [Right Ctrl] to speak.[/bold
yellow]")
                else:
                    console.print("\n[bold green]🎤 [MEDIA PAUSED/IDLE] Hands-Free Mode Active. Speak freely.[/bold green]")
        except Exception:
            pass
        await asyncio.sleep(1.0)

async def send_audio(session):
    global model_is_speaking, media_is_playing_state
    mic_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_IN,
        input=True,
        frames_per_buffer=CHUNK,
    )
    console.print("[cyan]🎤 [LISTENING] Dynamic Adaptive Audio Pipeline Online...[/cyan]")

    silence_frame = b'\x00' * (CHUNK * 2)

    try:
        while True:
            data = await asyncio.to_thread(
                mic_stream.read, CHUNK, exception_on_overflow=False
            )

            # Drop mic completely while JARVIS is responding
            if model_is_speaking:
                await session.send_realtime_input(
                    audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                )
                continue

            # State 1: Media is actively playing -> Require holding Right Ctrl
            if media_is_playing_state:
                if keyboard.is_pressed("right ctrl"):
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                    )
            # State 2: No media is playing -> 100% Hands-Free voice mode
            else:
                rms = calculate_rms(data)
                if rms > 450:
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                    )

    except asyncio.CancelledError:
        raise
    finally:
```

```
        try:
            mic_stream.stop_stream()
            mic_stream.close()
        except OSError:
            pass

async def receive_audio(session, audio_queue):
    global model_is_speaking
    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        await asyncio.to_thread(play_sfx_chime)

                        result = ""
                        if call.name == "open_website":
                            target_url = call.args.get("url", "https://google.com")
                            result = await asyncio.to_thread(open_website, target_url)
                        elif call.name == "open_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(open_application, str(target_app).strip())
                        elif call.name == "close_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(close_application, str(target_app).strip())
                        elif call.name == "purge_background_tasks":
                            result = await asyncio.to_thread(purge_background_tasks)
                        elif call.name == "control_media":
                            action = call.args.get("action", "play_pause")
                            result = await control_media(action)
                        elif call.name == "get_current_media_info":
                            result = await get_current_media_info()
                        elif call.name == "navigate_shorts":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(navigate_shorts, direction)
                        elif call.name == "skip_video":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(skip_video, direction)
                        elif call.name == "seek_video":
                            action = call.args.get("action", "forward")
                            result = await asyncio.to_thread(seek_video, action)
                        elif call.name == "toggle_video_view":
                            mode = call.args.get("mode", "fullscreen")
                            result = await asyncio.to_thread(toggle_video_view, mode)
                        elif call.name == "set_volume":
                            level = call.args.get("level", 70)
                            result = await asyncio.to_thread(set_volume, level)
                        elif call.name == "get_volume":
                            result = await asyncio.to_thread(get_volume)
                        elif call.name == "set_brightness":
                            level = call.args.get("level", 50)
                            result = await asyncio.to_thread(set_brightness, level)
                        elif call.name == "get_brightness":
                            result = await asyncio.to_thread(get_brightness)
                        elif call.name == "get_battery_status":
                            result = await asyncio.to_thread(get_battery_status)
                        elif call.name == "get_system_vitals":
                            result = await asyncio.to_thread(get_system_vitals)
                        elif call.name == "get_system_temperatures":
                            result = await asyncio.to_thread(get_system_temperatures)
                        elif call.name == "analyze_screen":
                            prompt = call.args.get("prompt", "Analyze what is visible on my screen.")
                            result = await analyze_screen(prompt)
                        elif call.name == "find_ui_element":
                            element_name = call.args.get("element_name", "")
                            result = await find_ui_element(element_name)

                        console.print(f"\n[bold yellow]⚙️ [TOOL EXECUTED: {call.name}][/bold yellow] [green]{result}[/green]")
                        function_responses.append(
                            types.FunctionResponse(
                                name=call.name,
                                id=call.id,
                                response={"result": result},
                            )
                        )

                    if function_responses:
                        await session.send_tool_response(function_responses=function_responses)

                server_content = response.server_content
```

```
                if server_content is not None:
                    if server_content.interrupted:
                        while not audio_queue.empty():
                            audio_queue.get_nowait()
                        model_is_speaking = False

                    if server_content.input_transcription:
                        console.print(f"\r[bold white][USER][/bold white] {server_content.input_transcription.text}")

                    model_turn = server_content.model_turn
                    if model_turn is not None:
                        if not model_is_speaking:
                            model_is_speaking = True
                            console.print("[bold cyan]🔊 [JARVIS ANSWERING...][/bold cyan]")

                        for part in model_turn.parts:
                            if part.inline_data and part.inline_data.data:
                                await audio_queue.put(part.inline_data.data)

                    if server_content.turn_complete:
                        model_is_speaking = False
                        console.print("[cyan]🎤 [LISTENING] Awaiting instructions...[/cyan]")

    except asyncio.CancelledError:
        raise
    finally:
        await audio_queue.put(None)

async def play_audio(audio_queue):
    speaker_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_OUT,
        output=True,
        frames_per_buffer=CHUNK,
    )
    try:
        while True:
            audio_data = await audio_queue.get()
            if audio_data is None:
                break
            await asyncio.to_thread(speaker_stream.write, audio_data)
    except asyncio.CancelledError:
        raise
    finally:
        try:
            speaker_stream.stop_stream()
            speaker_stream.close()
        except OSError:
            pass

async def run_session(config, model_id, greeting):
    async with client.aio.live.connect(model=model_id, config=config) as session:
        console.print("[bold green]✔ Secure Neural Bridge Established with Gemini Live.[/bold green]")

        restore_master_volume(70)
        await asyncio.to_thread(play_startup_chime)

        audio_queue = asyncio.Queue()
        monitor_task = asyncio.create_task(monitor_playback_state())
        send_task = asyncio.create_task(send_audio(session))
        receive_task = asyncio.create_task(receive_audio(session, audio_queue))
        playback_task = asyncio.create_task(play_audio(audio_queue))

        startup_prompt = f"Say aloud: 'All protocols initialized. {greeting}'"
        await session.send_realtime_input(text=startup_prompt)

        try:
            await asyncio.gather(monitor_task, send_task, receive_task, playback_task)
        finally:
            monitor_task.cancel()
            send_task.cancel()
            receive_task.cancel()
            playback_task.cancel()
            await asyncio.gather(monitor_task, send_task, receive_task, playback_task, return_exceptions=True)

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    greeting = get_time_greeting()

    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        realtime_input_config=types.RealtimeInputConfig(
            automatic_activity_detection=types.AutomaticActivityDetection(
                prefix_padding_ms=100,
                silence_duration_ms=300,
            )
        ),
        input_audio_transcription=types.AudioTranscriptionConfig(),
        speech_config=types.SpeechConfig(
```

```
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(
                    voice_name="Puck"
                )
            )
        ),
        system_instruction=types.Content(
            parts=[
                types.Part.from_text(
                    text=(
                        "You are JARVIS, an advanced artificial intelligence assistant created by "
                        "Swayam Dhawale, Founder of Pulse of AI. "
                        "Accent and Persona: Speak in an articulate, natural, polite Indian English accent. "
                        "Calm, highly sophisticated, and articulate, exactly like Tony Stark's JARVIS. "
                        "Always address the user as 'sir'. "
                        "Tool Execution Rules: "
                        "1. When opening any website or URL, call open_website and say: 'Yes sir, opening [site name]'. "
                        "2. When opening desktop apps, games, or tools, call open_application and say: 'Yes sir, opening [app name]'.
"
                        "3. When asked to close, shut down, terminate, or kill ANY app, browser, window, or game, call
close_application with the app name and say: 'Yes sir, closing [app name]'. "
                        "4. When asked to kill hidden background tasks, purge background processes, clean orphan threads, or optimize
RAM, call purge_background_tasks and report the memory freed with 'sir'. "
                        "5. When asked what song, video, or track is playing, or what media is active, call get_current_media_info
and announce the exact title with 'sir'. "
                        "6. When asked to pause, play, resume music or video, skip track, previous track, or mute/unmute audio, call
control_media with the action ('play_pause', 'next', 'previous', 'stop', 'mute') and say a crisp confirmation like 'Yes sir, pausing
playback' or 'Resuming playback, sir'. "
                        "7. When asked to scroll shorts, go to next short, or next reel, call navigate_shorts with direction='next'
and say: 'Yes sir, skipping to next Short'. When asked for previous short, call navigate_shorts with direction='previous'. "
                        "8. When asked for next video, skip to next video in playlist, or next YouTube video, call skip_video with
direction='next' and say: 'Yes sir, playing next video'. When asked for previous video, call skip_video with direction='previous'. "
                        "9. When asked to fast forward, skip 10 seconds ahead, or skip forward, call seek_video with
action='forward'. When asked to rewind 10 seconds, go back, or rewind, call seek_video with action='rewind'. "
                        "10. When asked to toggle full screen, enter full screen, or exit full screen, call toggle_video_view with
mode='fullscreen'. When asked for theater mode, call toggle_video_view with mode='theater'. "
                        "11. When setting volume, call set_volume and say: 'Yes sir, setting volume to [level]%'. "
                        "12. When asked what the volume level is, call get_volume and state the percentage crisply with 'sir'. "
                        "13. When setting brightness, call set_brightness and say: 'Yes sir, setting brightness to [level]%'. "
                        "14. When asked what the brightness is, call get_brightness and state the status crisply with 'sir'. "
                        "15. When asked about battery, power, or charging state, call get_battery_status and state the status with
'sir'. "
                        "16. When asked for temperatures, heat, thermal readings, or how hot the rig is running, call
get_system_temperatures and state the reading with 'sir'. "
                        "17. When asked for full diagnostics, performance vitals, system status, or complete CPU/GPU/RAM stats, call
get_system_vitals and give a short, high-tech summary. "
                        "18. When asked to inspect the screen, read an error, examine the terminal, or explain what is visible on
display, call analyze_screen. "
                        "19. When asked to locate or find a button, skip ad icon, or UI element on screen, call find_ui_element with
the name of the target element. "
                        "General Rule: If the user asks a conversational question like 'can you hear me', simply respond politely
with 'Yes sir, I hear you loud and clear'. Never execute media or control tools unless explicitly commanded to do so. "
                        "Multilingual Rule: Support over 70 global languages. Detect the user's spoken language and respond fluently
in that exact same language while keeping the polite 'sir' honorific. "
                        "Respond in short, crisp sentences. Never speak markdown or bullet points. "
                        "If asked about your identity or creator, state clearly that you were built by Swayam Dhawale."
                    )
                )
            ]
        ),
    )

    console.print("[bold cyan]═══════════════════════════════════════════════════════════════[/bold cyan]")
    console.print("[bold white] ⚡ JARVIS MULTIMODAL CORE - EPISODE 14 (LOCAL GPU VISION & GROUNDING) ⚡ [/bold white]")
    console.print("[bold yellow] Built by Swayam Dhawale, Founder of Pulse of AI.[/bold yellow]")
    console.print("[bold cyan]═══════════════════════════════════════════════════════════════[/bold cyan]")

    while True:
        try:
            await run_session(config, model_id, greeting)
        except (
            errors.APIError,
            websockets.exceptions.ConnectionClosedError,
            ConnectionResetError,
            OSError,
        ) as e:
            console.print(f"\n[bold red]🔄 Connection dropped ({e}). Reconnecting in 1 second...[/bold red]")
            await asyncio.sleep(1)

if __name__ == "__main__":
    if sys.platform == "win32":
        asyncio.set_event_loop_policy(asyncio.WindowsProactorEventLoopPolicy())
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        console.print("\n[bold red]🛑 JARVIS session terminated by user.[/bold red]")
    finally:
        p.terminate()
```